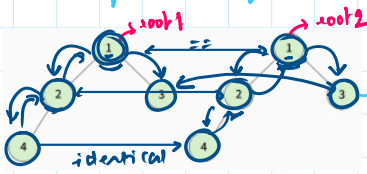
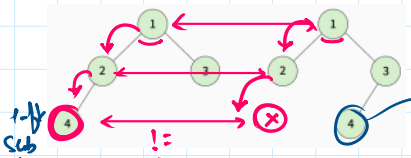
① identical trees (Binary Trees)

i/p → 2 B.T

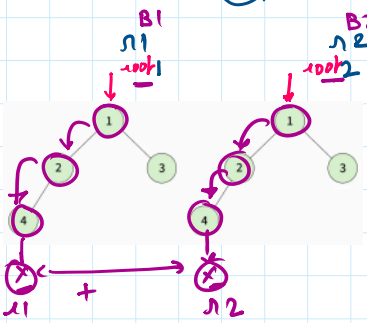
o/p → true, false.

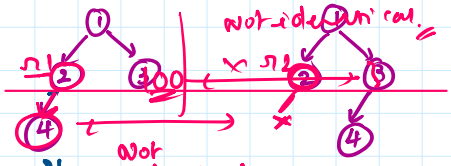


return true; → B.T is identical.



return false → this tree is not identical;

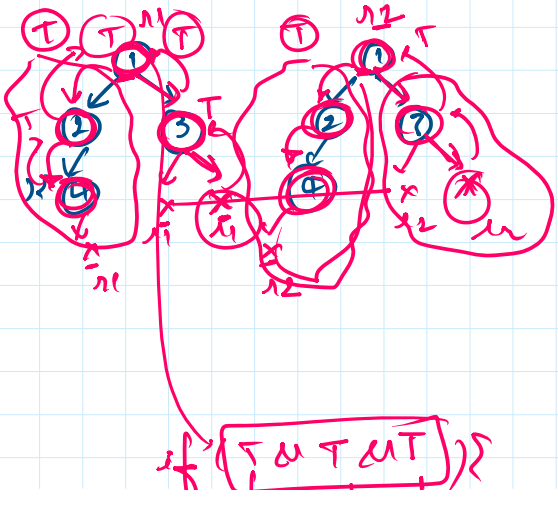


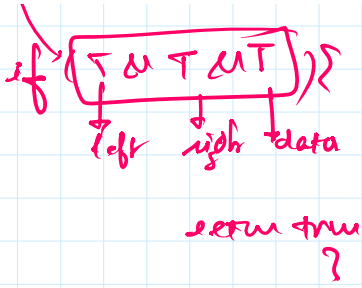


if(n1!=NULL && n2==NULL) return false;

```
bool isIdentical(n1, n2) {
    if(root==NULL){
        return NULL/true;
    }
    → if(n1==NULL && n2==NULL)
        return true;
    }
    * if(n1!=NULL && n2==NULL)
        return false;
    }
    A if(n1==NULL && n2!=NULL)
        return false
}
```

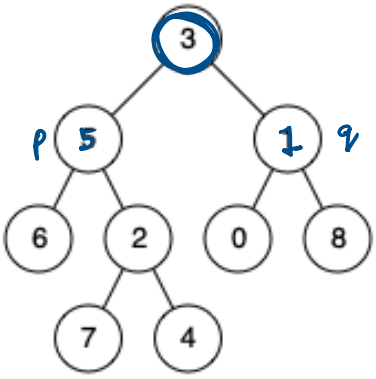
```
bool left  = isIdentical(n1→left, n2→left);
bool right = isIdentical(n1→right, n2→right);
bool data  = (n1→data == n2→data)
if(left && right && data) true
    return true;
}
return false;
```





Lowest Common ancestor

B.T.

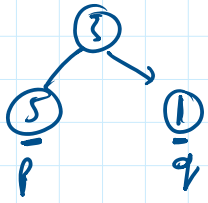


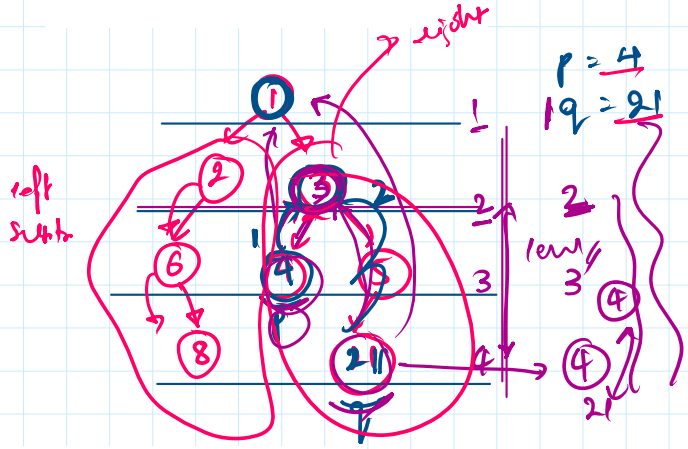
p = 5

q = 1

op = 3

Common Ancestor

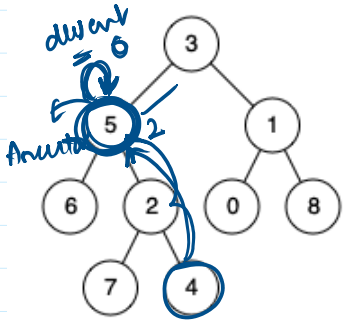




** lowest common

Ans = 3

```
if(p==root->data || q==root->data)
    return true
```

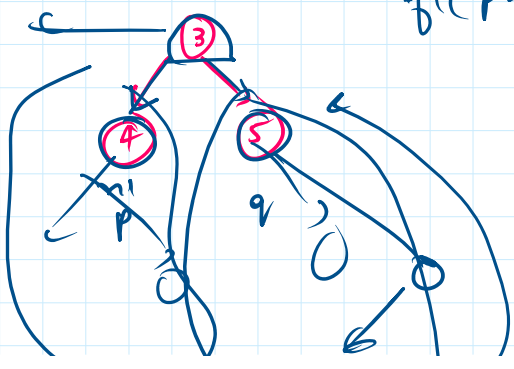


p = 5 } child node

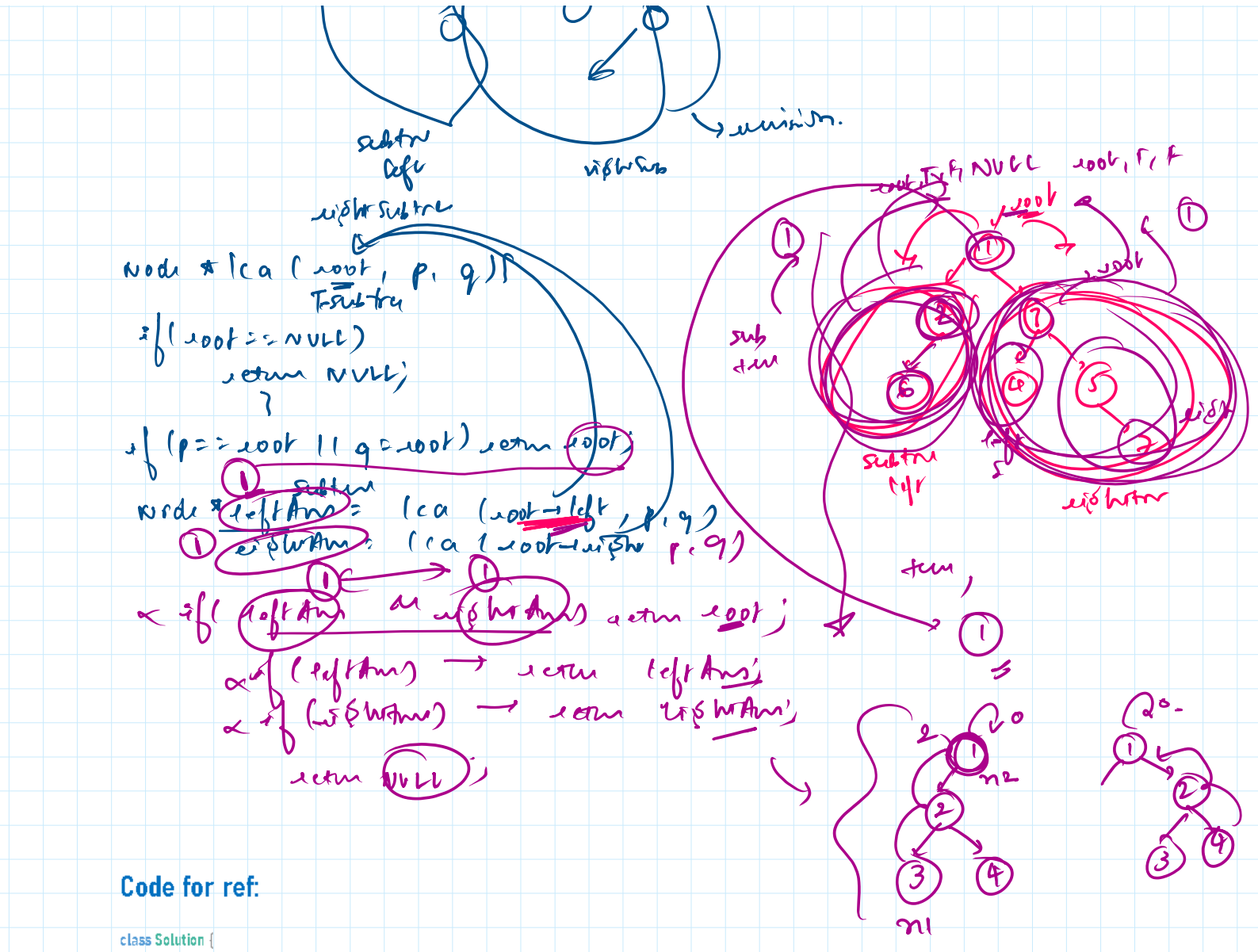
q = 4

LCA → 5

LCA




```
if((p==root->left) || (q==root->right))
    return root;
```



Code for ref:

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if(root == NULL){
            return NULL;
        }
        if(p == root || q == root){
            return root;
        }
        TreeNode* leftSide = lowestCommonAncestor(root->left, p, q);
        TreeNode* rightSide = lowestCommonAncestor(root->right, p, q);
        if(leftSide && rightSide){
            return root;
        }
        if(leftSide){
            return leftSide;
        }
        if(rightSide){
            return rightSide;
        }
        return NULL;
    }
```